NO PRUMO

Cadastrando um item novo no sistema

Do modelo de dados até o endpoint — e as instruções prontas para uma IA construir a tela.

Este guia descreve o caminho de um cadastro novo: Funcionários, Clientes, Fornecedores, Obras. O exemplo usado o tempo todo é Funcionários (Employee), mas a receita é a mesma para qualquer um.

O cadastro de Usuários já está pronto no repositório e é a referência viva: quando este documento e o código discordarem, o código vence. Leia UsersController.cs antes de começar.

Atualizado em 17/09/2026.

O caminho inteiro, em seis passos

1. Conferir a entidade — ela provavelmente já existe em 04-Domain/Entities.
2. Criar os DTOs em 02-Application/DTOs: um de saída e os de entrada.
3. Declarar a permissão no DatabaseSeeder e distribuí-la nos papéis.
4. Registrar a policy no Program.cs.
5. Escrever o controller em 01-Presentation/Controllers.
6. Testar no Swagger e só então pedir a tela para a IA.

> **A ORDEM IMPORTA**
> Cada passo depende do anterior. Escrever o controller antes da permissão significa um endpoint aberto; pedir a tela antes de testar o Swagger significa depurar duas camadas ao mesmo tempo, sem saber qual das duas está errada.

Passo 1 — A entidade

As 36 entidades do sistema já existem em back/04-Domain/Entities, geradas a partir do modelo de dados. Na maioria dos cadastros não há nada a criar aqui: abra o arquivo, confira os campos e siga.

Duas regras valem sempre. O código é escrito em inglês — classe, propriedade e valor gravado (status = "pending", nunca "pendente"). E o nome da coluna no banco é consequência do nome em C#: a conversão para snake_case é automática, feita pelo UseSnakeCaseNamingConvention no Program.cs. Renomear uma propriedade renomeia a coluna.

Se faltar uma coluna, declare no AppDbContext e deixe o comando gerar a migration:

```
É UMA LINHA SÓ
dotnet ef migrations add NomeDaMudanca --project back/03-Infrastructure
  --startup-project back/01-Presentation
```

> **MIGRATION NUNCA SE EDITA À MÃO**
> O arquivo gerado não recebe uma linha de SQL escrita por pessoa nenhuma. Se algo precisa existir no banco — valor padrão, restrição, seed — declare no AppDbContext (HasDefaultValue, HasCheckConstraint, HasData) e gere de novo pelo comando.

Isso tem um preço que o código paga: o banco não tem CHECK constraints nem valores padrão nas colunas numéricas. Validar faixa, data e obrigatoriedade é trabalho da entidade ou do controller.

Passo 2 — Os DTOs

Entidade não vai para a tela. Os DTOs ficam em back/02-Application/DTOs, um arquivo por tipo, e existem em três sabores: o de saída, o de criação e o de edição.

02-APPLICATION/DTOS/EMPLOYEEDTO.CS

```
public sealed record EmployeeDto(
    long Id,
    string Name,
    string JobRoleName,
    bool Active)
{
    public static EmployeeDto FromEntity(Employee e) => new(
        e.Id, e.Name, e.JobRole.Name, e.Active ?? true);
}
public sealed record CreateEmployeeRequest(string Name, long JobRoleId);
public sealed record UpdateEmployeeRequest(string Name, long JobRoleId);
```

Repare no que o DTO não tem: nenhuma propriedade para campo sensível. É assim que um dado não vaza por esquecimento — se a propriedade não existe, não há como preenchê-la sem querer.

> **MESTRE ENXERGA QUANTIDADE. ADMIN ENXERGA DINHEIRO.**
> O mestre de obras nunca vê valor de contrato, custo de mão de obra, salário, preço de material ou margem. Quando um recurso tem campo de dinheiro, ele ganha DOIS DTOs — um por perfil. Nunca um DTO único com os campos anulados para o mestre: alguém adiciona um campo novo, esquece de anular, e vaza. Esconder na tela não resolve, porque o JSON continua indo pelo DevTools.

Para listas, reaproveite o PagedResult<T> que já existe. Ele carrega items, page, size, total e totalPages, e o front já sabe ler esse formato.

Passo 3 — A permissão

A autorização do sistema pergunta o que a pessoa pode fazer, nunca quem ela é. Não existe if (role == "admin") em lugar nenhum do código — existe permissão, e o papel é só um apelido para um conjunto delas.

Cada recurso nasce com dois verbos, um para ler e outro para escrever. É o que permite um perfil acompanhar uma tela sem poder mexer nela, sem precisar inventar um papel novo:

PERMISSIONCATALOG, EM 03-INFRASTRUCTURE/DATA/DATABASESEEDER.CS

```
("view_employees",   "Ver funcionários"),
("manage_employees", "Cadastrar e editar funcionários"),
```

E depois distribua nos papéis que recebem cada um, no RoleCatalog do mesmo arquivo. O admin recebe todos automaticamente.

> **NÃO PRECISA DE MIGRATION**
> Permissão é dado, não estrutura. O seeder roda a cada start da API e insere só o que falta, então basta subir a API de novo. Quem já estiver logado precisa sair e entrar: as permissões viajam dentro do token.

Passo 4 — A policy

01-PRESENTATION/PROGRAM.CS, DENTRO DE ADDAUTHORIZATION

```
options.AddPolicy("view_employees",
    policy => policy.RequireClaim("permission", "view_employees"));
options.AddPolicy("manage_employees",
    policy => policy.RequireClaim("permission", "manage_employees"));
```

Passo 5 — O controller

O molde está em UsersController.cs. A estrutura, reduzida ao essencial:

```
[ApiController]
[Route("api/[controller]")]
[Authorize(Policy = "view_employees")]   // o mínimo para abrir a tela
public sealed class EmployeesController(AppDbContext db) : ControllerBase
{
    private const int MaxPageSize = 100;
    [HttpGet]
    public async Task<ActionResult<PagedResult<EmployeeDto>>> List(
        [FromQuery] int page = 1,
        [FromQuery] int size = 10,
        [FromQuery] string? search = null)
    { ... }
    [HttpPost]
    [Authorize(Policy = "manage_employees")]   // escrever exige o outro verbo
    public async Task<ActionResult<EmployeeDto>> Create(CreateEmployeeRequest r)
    { ... }
}
```

As regras que o molde carrega

- Paginação com trava: page = Math.Max(1, page) e size = Math.Clamp(size, 1, MaxPageSize). Os dois vêm da query string, ou seja, vêm de fora — um size=100000 derruba a consulta.
- Contar antes de paginar: o total é do filtro inteiro, não da página que está sendo devolvida.
- AsNoTracking em toda leitura. Sem isso o EF guarda estado que ninguém vai usar.
- Erro de campo volta como ValidationProblem, com o nome do campo. É o que faz a mensagem aparecer embaixo do campo certo na tela, em vez de um alerta genérico no topo.
- Desativar em vez de apagar. Cadastro tem histórico: PATCH /{id}/activate e /{id}/deactivate, nunca DELETE.
- Quando o registro não é do escopo da pessoa, responda 404, não 403. O 403 confirma que aquele id existe e deixa enumerar.

Passo 6 — Testar antes de pedir a tela

Suba a API e abra http://localhost:5262/swagger. Faça login em /api/auth/login, copie o token, clique em Authorize no topo e cole. A partir daí dá para exercitar cada endpoint novo.

Vale testar também o caminho negativo: entre com um usuário que não tenha o verbo de escrita e confirme que o POST responde 403. Se responder 200, a policy não foi aplicada.

Regras de negócio que não se negociam

Valem para qualquer cadastro novo. Não são preferências de estilo — são decisões de produto já fechadas.

ASSUNTO	A REGRA
Dinheiro	O mestre de obras não vê valor, custo, salário, preço nem margem. DTOs separados por perfil, nunca campo anulado.

Saldo de estoque	Nunca sai de uma coluna guardada. É sempre somado de stock_movement, que é o razão da movimentação.
Atrasada	Não é um status gravado. É calculado comparando a data prevista com hoje, em C#.
Cliente	Não tem login. Acessa por link com token, em project_link.
Empreiteiro	Não tem login. É fornecedor, não usuário do sistema.
Cadastro de gente	Não existe auto-cadastro. O admin cria a conta e entrega uma senha provisória.
Compras	O mestre solicita informando só quantidade. Preço e nota fiscal são da administração.

Checklist do back

- A entidade existe e tem todos os campos de que a tela precisa.
- Os DTOs não têm nenhuma propriedade de campo sensível.
- As duas permissões estão no seeder e distribuídas nos papéis.
- As duas policies estão registradas no Program.cs.
- O controller tem o verbo de leitura na classe e o de escrita nas ações que escrevem.
- A listagem devolve PagedResult e tem trava de tamanho de página.
- Testado no Swagger, inclusive o 403 de quem não tem o verbo de escrita.

Instruções para a IA construir o front

Com o back testado, a tela é a parte mecânica. O texto do quadro abaixo pode ser colado direto numa conversa com uma IA que tenha acesso ao repositório — ele descreve o padrão do projeto e os arquivos a criar.

Troque Employee, employees e Funcionários pelo recurso que você está construindo.

TEXTO PARA COLAR NA IA

```
Construa a tela de Funcionários no front, seguindo o padrão que já existe
na tela de Usuários. Leia estes arquivos antes de escrever qualquer coisa:
  front/app/view/screens/users/UsersScreen.jsx
  front/app/view/screens/users/UserFormDialog.jsx
  front/app/data/gateways/userGateway.js
  front/app/container.js
  front/app/view/routes.jsx
  front/app/view/shell/AppShell.jsx
O contrato real dos endpoints está em
back/01-Presentation/Controllers/EmployeesController.cs. Confira lá em vez
de supor o formato da resposta, e não invente endpoint que não existe.
Crie:
1. front/app/data/gateways/employeeGateway.js, no mesmo formato do
   userGateway: uma função createEmployeeGateway(http) devolvendo os
   métodos, com o contrato dos endpoints documentado no comentário do topo.
2. Registre o gateway em front/app/container.js.
3. front/app/view/screens/employees/EmployeesScreen.jsx, com listagem
   paginada, busca e os diálogos de criar e editar. Use os componentes que
   já existem em view/ui (TextField, SelectField, Button, Dialog,
   Pagination, Badge). Não crie componente de UI novo sem necessidade.
4. A rota em routes.jsx, irmã de /users, dentro do AppShell e protegida
   por <RequireSession permission="view_employees">.
5. O item de menu em AppShell.jsx, no grupo Master data, declarando
   permission: "view_employees".
Regras:
- Texto de tela em inglês. Código e nomes de arquivo em inglês.
- Sem mock e sem dado inventado: tudo vem da API.
- A tela nunca importa de data/ nem chama fetch. Ela recebe o gateway pelo
  contexto do container.
- Todo botão que escreve fica dentro de can("manage_employees"), para que
  quem só tem view_employees veja a tela sem os botões de escrita.
- Todo campo tem <label>. Placeholder complementa, nunca substitui.
- Erro de campo vem do ProblemDetails e aparece embaixo do campo,
  não num alerta no topo.
```

Por que a tela recebe o gateway em vez de chamar a API direto

O container.js é o único arquivo que sabe como o aplicativo está ligado: onde fica a API, onde mora o token, como um erro vira mensagem. A tela pede o gateway e usa. Se amanhã o endereço da API mudar, ou o token passar a vir de outro lugar, muda um arquivo — e nenhuma tela é aberta.

O que conferir quando a IA terminar

- A tela carrega dados reais da API, não uma lista escrita no código.
- Entrar com um usuário que só tem o verbo de leitura mostra a tela sem os botões de escrita.
- Salvar com um campo inválido mostra a mensagem embaixo do campo certo.
- O item aparece no menu no grupo certo, e some para quem não tem a permissão.
- A paginação funciona com mais registros do que cabem numa página.
- Nenhum import de data/ dentro de uma tela, e nenhum fetch solto.

> **O MENU AGRUPADO NÃO É ENFEITE**
> Funcionários, Clientes e Fornecedores são todos cadastros, então entram sob o mesmo grupo de Usuários, e não soltos na raiz do menu. A forma do menu é o que ensina a próxima pessoa onde a tela dela pertence.

Se faltar alguma coisa no back

Acontece de a tela precisar de um campo que o endpoint não devolve, ou de um endpoint que não existe. Nesse caso a regra é parar e avisar a equipe, não implementar o outro lado por conta própria — quem mantém o back precisa saber o que entrou nele.

O mesmo vale para a IA: se ela disser que precisa mexer no back para terminar a tela, é sinal de que o passo 5 deste guia não ficou completo. Volte, ajuste o controller, teste no Swagger e só então peça a tela de novo.